

# KCPC Week 3 Papers

## Foundations of Lambda Calculus and Computation

King's Competitive Programming Club  
Curated by Mehmet Kutay Bozkurt (@mkutay)

An annotated guide to the original research behind Workshop 3. Each entry gives a complete citation, a stable location and the part relevant to the slides and exercises. For a gentler first pass use *Week 3 Reading*; the 1930s papers are primary sources, not introductory tutorials.

### I. Computation and lambda-definability

#### [1] Church, A. (1936)

*An unsolvable problem of elementary number theory.* American Journal of Mathematics, 58(2), 345–363. <https://doi.org/10.2307/2371045>

**Where / why:** Pages 345–349 motivate effective calculability and lambda-definability; §§2–4 set up the formal apparatus. This is the historical source for treating function abstraction and application as a model of computation (slides: overview and purity). The undecidability results are beyond this workshop.

#### [2] Turing, A. M. (1936–1937)

*On computable numbers, with an application to the Entscheidungsproblem.* Proceedings of the London Mathematical Society, s2-42(1), 230–265. <https://doi.org/10.1112/plms/s2-42.1.230>

**Where / why:** §1 defines machine computation; the later discussion of computable numbers and the Entscheidungsproblem supplies the parallel machine-based notion. Read alongside Church (1936) for context for the slide's statement about describing computation, rather than as a claim that a computer literally has no numbers.

### II. Reduction and substitution

#### [3] Church, A., & Rosser, J. B. (1936)

*Some properties of conversion.* Transactions of the American Mathematical Society, 39(3), 472–482. <https://doi.org/10.2307/1989762>

**Where / why:** Pages 472–476 introduce conversion and its central consistency properties. The Church–Rosser property shows that two reduction paths can be reconciled; when a beta-normal form exists it is unique up to alpha-renaming. This underpins the slides' beta-reduction rule and the worksheet's reduction proofs.

#### [4] de Bruijn, N. G. (1972)

*Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem.* Indagationes Mathematicae, 34(5), 381–392 (also Proceedings, Series A, 75(5)). [https://doi.org/10.1016/1385-7258\(72\)90034-0](https://doi.org/10.1016/1385-7258(72)90034-0)

**Where / why:** Opening pages 381–383 replace bound-variable names with indices recording binder distance, removing accidental name clashes. This addresses the capture pitfall in the slides’ alpha/beta formulas and Problem 3 and motivates safe substitution in interpreters.

### III. Encodings and types

#### [5] Church, A. (1940)

*A formulation of the simple theory of types.* The Journal of Symbolic Logic, 5(2), 56–68.  
<https://doi.org/10.2307/2266170>

**Where / why:** Pages 56–58 introduce types for function application and abstraction. This contrasts the workshop’s *untyped* pure encodings of Boolean selectors and Church numerals with a typed formal system; useful background for later functional programming workshops.

### IV. One argument at a time

#### [6] Schönfinkel, M. (1924)

*Über die Bausteine der mathematischen Logik.* Mathematische Annalen, 92(3–4), 305–316.  
<https://doi.org/10.1007/BF01448013>

**Where / why:** Pages 305–308 explain how multi-argument expressions can be represented by successive single-argument applications; the later pages develop combinators. This is a foundational source for the slides’ currying section and selector exercises. An English translation, *On the Building Blocks of Mathematical Logic*, appears in Jean van Heijenoort (ed.), *From Frege to Gödel* (Harvard University Press, 1967), pp. 355–366.

For elementary definitions and reduction arguments for Boolean selectors and Church numeral arithmetic, see Selinger’s *Lecture Notes on the Lambda Calculus*, §§3.1–3.2. The supplied Ker lecture notes introduce Boolean selectors and numerals in Chapter 4, §4.1, printed pp. 43–44; Exercises 4.1–4.2, printed p. 50, ask readers to construct AND and verify a predecessor, including its zero case. See *Week 3 Reading* for the links. These lecture notes are secondary expositions, so they are listed in the reading guide rather than treated as original research papers.